Especificación del Motor Sargon Adaptativo

Aprendizaje neuronal de valor posicional, valor contextual de piezas y profundidad de análisis

Especificación de trabajo

29 de septiembre de 2026

Resumen

Este documento especifica una evolución gradual del algoritmo SARGON mediante tres redes neuronales especializadas: una red de valor posicional, una red de valor contextual de las piezas y una red de profundidad de análisis. La arquitectura se diseña con dos prioridades: preservar el comportamiento de SARGON como línea base cuando no existe aprendizaje efectivo, y hacer que todo progreso sea observable, medible y auditable.

La especificación define la interacción entre las fuentes de juego UCI, `learn`, workers y archivos PGN; establece un mecanismo uniforme de backpressure por productor; fija la coherencia de un único modelo durante toda una partida; determina cómo se aprende a partir de análisis realizados, resultados de partida y jugadas expertas de PGN; y establece las métricas mínimas que deben registrarse para demostrar si el sistema aprende realmente.

El objetivo explícito es evitar arquitecturas opacas en las que el volumen de datos, la concurrencia o la complejidad de la infraestructura oculten un aprendizaje débil o inexistente.

Índice

1. Propósito y alcance

El propósito de esta especificación es definir un motor de ajedrez adaptativo basado en SARGON que:

1. utilice SARGON clásico como conocimiento inicial;
2. introduzca aprendizaje exclusivamente mediante redes neuronales;
3. mantenga separadas tres capacidades aprendidas: valor posicional, valor contextual de las piezas y profundidad necesaria de análisis;
4. pueda aprender de partidas UCI, partidas generadas mediante `learn`, partidas producidas por workers y archivos PGN;
5. trate todo PGN importado como experiencia experta;
6. mantenga un modelo coherente durante una partida completa;
7. impida que cualquier productor comience una nueva partida hasta que la anterior haya sido incorporada al aprendizaje;
8. disponga de métricas suficientes para determinar si cada red aprende, cuánto aprende y si ese aprendizaje mejora el comportamiento del motor.

Queda fuera del alcance de esta versión:

- aprendizaje directo mediante tablas persistentes por posición exacta;
- uso del estado histórico de una posición como conocimiento operativo principal;
- redes de política separadas;
- MCTS o variantes tipo AlphaZero;
- actualización directa de redes desde UCI, workers o `learn`;
- escritura concurrente sobre los parámetros del modelo.

2. Principios de diseño

2.1. Sargon como línea base

La primera propiedad requerida es:

$$\boxed{\text{sin aprendizaje efectivo} \Rightarrow \text{SARGON clásico}}$$

Sea $V_S(s)$ la evaluación clásica de SARGON para una posición s. El sistema debe comenzar cumpliendo

$$V_{\text{eff}}(s) = V_S(s).$$

Análogamente, el valor contextual inicial de cada pieza debe coincidir con el valor clásico y la profundidad inicial con la profundidad base de SARGON.

2.2. Aprendizaje neuronal exclusivo

Toda adaptación se realizará mediante tres redes neuronales:

$$N_V, \qquad N_P, \qquad N_D.$$

Sus tamaños serán configurables para permitir experimentación controlada.

2.3. Separación de responsabilidades

Se establece la regla:

$$\boxed{\text{los productores generan experiencia; sólo el learner modifica el modelo}}$$

Los productores son UCI, `learn`, cada worker y el importador PGN. Ninguno actualiza pesos neuronales directamente.

2.4. Coherencia por partida

Toda partida debe jugarse utilizando un único snapshot del modelo:

$$\mathcal{M}_k.$$

Una partida que comienza con $\mathcal{M}_k$ termina con $\mathcal{M}_k$. No se permite reemplazar el modelo en mitad de la partida.

2.5. Backpressure por productor

Cada productor queda bloqueado para iniciar una nueva partida hasta recibir confirmación de que su partida anterior fue incorporada al aprendizaje. La espera es independiente por productor:

$$\boxed{G_i \to \text{learn}(G_i) \to ACK(G_i) \to G_{i+1}}$$

3. Modelo global

Una versión del conocimiento operativo se define como

$$\mathcal{M}_k = \{N_V^{(k)}, N_P^{(k)}, N_D^{(k)}, \Theta_k, \mathcal{H}_k\},$$

donde Θ_k contiene parámetros de moderación y $\mathcal{H}_k$ metadata, métricas y versión. Cada partida almacena la versión de modelo que la produjo.

4. Red de valor posicional

4.1. Objetivo

La ValueNet aprende la corrección posicional que Sargon no representa adecuadamente:

$$N_V(x_V(s)) = \Delta V_{\text{pos}}(s).$$

La evaluación efectiva será

$$V_{\text{eff}}(s) = V_S(s) + \mu_V \Delta V_{\text{pos}}(s) + \Delta V_{\text{piece}}(s),$$

donde μ_V es un moderador.

4.2. Entrada

$x_V(s)$ deberá contener una representación normalizada y versionada de, al menos: diferencia material, movilidad, control del centro, desarrollo, seguridad del rey, estructura de peones, peones pasados, actividad de piezas, espacio, iniciativa, lado al turno y fase de partida.

4.3. Arquitectura configurable

Ejemplos:

```
--value-net 32
--value-net 64,32
--value-net 128,64,32
--value-net 256,128,64
```

La arquitectura usada se persistirá con el modelo.

4.4. Target de entrenamiento

Para una posición s_t, sean $V_S(s_t)$ la evaluación estática clásica y $V_{d_t}(s_t)$ el resultado de la búsqueda realmente realizada a profundidad d_t. La búsqueda proporciona

$$Y_t^{\text{search}} = V_{d_t}(s_t).$$

El resultado posterior produce un retorno temporal Y_t^{TD}. El target combinado es

$$Y_t^V = \alpha_t Y_t^{\text{search}} + (1 - \alpha_t) Y_t^{\text{TD}}.$$

La red aprende la corrección residual

$$\boxed{T_t^V = Y_t^V - V_S(s_t) - \Delta V_{\text{piece}}(s_t)}.$$

La pérdida básica es

$$L_V = \frac{1}{B} \sum_{t=1}^{B} \left(N_V(x_V(s_t)) - T_t^V \right)^2.$$

4.5. Retroalimentación por resultado

No se asignará directamente el resultado final a todas las posiciones. Sea

$$z \in \{-1, 0, +1\}.$$

Se utilizará aprendizaje temporal con

$$\delta_t = r_{t+1} + \gamma V_{\text{eff}}(s_{t+1}) - V_{\text{eff}}(s_t),$$

y TD(λ) para controlar la propagación de consecuencias tardías hacia posiciones anteriores. El retorno se calcula una vez terminada la partida.

5. Red de valor contextual de piezas

5.1. Objetivo

La PieceNet aprende cuánto difiere el valor efectivo de una pieza de su valor clásico en función de la posición:

$$N_P(x_P(s)) = (\Delta P, \Delta N, \Delta B, \Delta R, \Delta Q).$$

Los valores efectivos serán

$$v_X(s) = v_X^{(0)} + \mu_P \Delta X(s), \qquad X \in \{P, N, B, R, Q\}.$$

5.2. Separación semántica

La PieceNet sólo podrá intervenir en términos asociados directamente con valor contextual de piezas. La evaluación se descompone como

$$V_{\text{eff}}(s) = V_S(s) + \Delta V_{\text{piece}}(s) + \Delta V_{\text{pos}}(s).$$

Así se evita que ValueNet y PieceNet aprendan de manera indistinguible el mismo fenómeno.

5.3. Entrada

$x_P(s)$ contendrá como mínimo: tipo y cantidad de piezas, movilidad por tipo, centralización, control de casillas, puestos avanzados, columnas abiertas y semiabiertas, diagonales activas, actividad relativa, estructura de peones, fase de partida, seguridad de reyes y relaciones pieza-estructura.

5.4. Arquitectura configurable

```
--piece-net 32
--piece-net 64,32
--piece-net 128,64
```

La salida tendrá siempre cinco valores.

5.5. Aprendizaje

La PieceNet se entrena a partir del mismo target global de valor, pero el gradiente queda restringido a la contribución material contextual. El error global es

$$E_t = Y_t^V - V_{\text{eff}}(s_t).$$

6. Red de profundidad

6.1. Objetivo

La DepthNet aprende cuánto análisis adicional necesita una posición futura:

$$N_D(x_D(s)) = q(s).$$

La profundidad aplicada es

$$d_{\text{eff}}(s) = \text{clip}(d_0 + \Delta d(s), d_{\text{mín}}, d_{\text{máx}}).$$

6.2. Moderadores obligatorios

La salida neuronal se aplica siempre moderada:

$$\Delta d(s) = \text{round}(\mu_D N_D(x_D(s))).$$

Inicialmente

$$0 \leq \mu_D \leq 1, \qquad 0 \leq \Delta d(s) \leq \Delta d_{\text{máx}}.$$

En la etapa inicial la red puede agregar profundidad, pero no reducir la profundidad base.

6.3. Target de profundidad

Durante una búsqueda se conservan

$$V_1, V_2, \ldots, V_D.$$

Sea V_D el valor a la mayor profundidad alcanzada. Se define

$$d^* = \text{mín}\{d : |V_d - V_D| < \epsilon \text{ y la estabilidad se mantiene}\}.$$

La estabilidad deberá mantenerse durante un número configurable de profundidades posteriores. El target es

$$T^D = d^* - d_0,$$

y la pérdida básica

$$L_D = \frac{1}{B} \sum \left(N_D(x_D(s)) - T^D \right)^2.$$

6.4. Aplicación futura

El aprendizaje de profundidad no se guarda como regla por posición exacta. Queda codificado en N_D. Cuando una posición futura presenta características similares, la red puede solicitar análisis adicional antes de que SARGON descubra tardíamente su dificultad.

7. Fuentes de experiencia

7.1. UCI

UCI utiliza un snapshot fijo durante toda la partida. Al finalizar genera experiencia, la envía al learner, queda bloqueado y sólo acepta una nueva partida después del ACK.

7.2. Modo learn

`learn` sigue exactamente el mismo protocolo:

$$\boxed{\texttt{learn game} \rightarrow \text{learner} \rightarrow ACK \rightarrow \text{next learn game}}.$$

7.3. Workers

Cada worker es un productor independiente. El worker i ejecuta

$$G_i^k \rightarrow \text{learner} \rightarrow ACK_i \rightarrow G_i^{k+1}.$$

El ACK de un worker no bloquea a otros. Cada worker utiliza el último modelo disponible al iniciar su partida, mantiene ese modelo hasta el final, no entrena redes y produce experiencia adversarial.

7.4. PGN

Todo PGN importado se considera fuente experta:

$$\boxed{\text{PGN importado} = \text{experiencia experta}}.$$

No existirán categorías separadas de PGN genérico, maestro o engine.

8. Aprendizaje desde PGN

8.1. Preferencia experta

Para una posición s_t, sea a_t^* la jugada del PGN y $a_1, \ldots, a_n$ las jugadas legales. El sistema incorpora

$$Q(s_t, a_t^*) \gtrsim Q(s_t, a_j), \qquad a_j \neq a_t^*.$$

No se exige que a_t^* sea la única jugada buena; se trata como evidencia experta de preferencia.

8.2. Procesamiento

Al importar una partida se reconstruye cada posición, se extraen features para las tres redes, se ejecuta SARGON con el modelo vigente, se analiza la jugada experta, se comparan alternativas relevantes, se incorpora el resultado final y se generan targets para valor, piezas y profundidad.

8.3. Pérdida de preferencia

Se utilizará una pérdida de ranking

$$L_{\text{PGN}} = \sum_{a \neq a^*} \max\left(0, m - [Q(s, a^*) - Q(s, a)]\right),$$

con margen m configurable. La experiencia PGN tendrá un peso superior al de experiencia autocreada:

$$\omega_{\text{PGN}} > 1.$$

9. Representación de una partida aprendible

Cada partida producirá un objeto lógico:

```
GameExperience
    game_id
    source
    model_version
    result
    positions[]
```

Cada posición contendrá:

```
PositionExperience
    ply
    side_to_move
    features_value
    features_piece
    features_depth
    sargon_static_value
    effective_value
    search_depth
    search_values_by_depth[]
    best_move
    candidate_moves[]
        move
        value
        depth
    played_move
    expert_move # si source=PGN
```

Al terminar la partida se calculan los targets TD, de valor y de profundidad.

10. Ciclo de aprendizaje de una partida

```
productor toma Model Mk
        |
        v
 juega partida completa
        |
        +-- posicion 1: evaluacion + busqueda
        +-- posicion 2: evaluacion + busqueda
        +-- ...
        +-- posicion N
        |
        v
   resultado final
        |
        v
 GameExperience completo
        |
        v
      learner
        |
        +-- calcula retornos TD(lambda)
        +-- genera targets ValueNet
        +-- genera targets PieceNet
        +-- genera targets DepthNet
        +-- aplica preferencia PGN cuando corresponda
        +-- actualiza redes
        |
        v
    publica Model Mk+1
        |
        v
     ACK(game_id)
```

11. Definición de partida aprendida

Una partida se considera aprendida cuando:

1. su resultado final fue registrado;
2. sus posiciones fueron convertidas en ejemplos;
3. se calcularon los targets temporales;
4. se aplicaron los targets de análisis;
5. se ejecutó al menos una actualización de todas las redes habilitadas;
6. el modelo resultante fue publicado;
7. la experiencia quedó persistida;
8. se emitió el ACK al productor.

No se requiere entrenar múltiples epochs antes del ACK. La experiencia queda disponible para replay posterior.

12. Backpressure

La regla uniforme para UCI, `learn` y cada worker es

$$\boxed{\text{máximo una partida pendiente por productor}}.$$

Con W workers, un UCI y un `learn`, el máximo teórico de partidas finalizadas pendientes es aproximadamente $W + 2$. No existe una barrera global: el bloqueo es local a cada productor.

13. Snapshots y publicación de modelos

Al comenzar una partida:

$$local_model \leftarrow current_model.$$

Toda la partida utiliza ese objeto. El learner no modifica el modelo actual en el lugar: construye $\mathcal{M}_{k+1}$ y lo publica atómicamente. Los productores que ya juegan conservan $\mathcal{M}_k$; los que comienzan una nueva partida después del ACK toman la versión más reciente.

14. Persistencia

14.1. Objetivos

La persistencia sirve para guardar modelos, experiencia, replay, auditoría, rollback y reconstrucción de métricas. No es el mecanismo primario de decisión durante el juego.

14.2. Modelo

Debe persistirse, como mínimo:

```
model_version
created_at
value_net_arch
piece_net_arch
depth_net_arch
value_net_weights
piece_net_weights
depth_net_weights
moderators
training_counters
metrics
```

14.3. Experiencia

```
game_id
source
model_version
result
positions
targets
learned_at
```

14.4. Reuso del conocimiento

No se utilizará una posición histórica exacta como fuente primaria de decisión futura. El conocimiento reutilizado se encuentra en N_V, N_P, N_D:

$$\boxed{\text{experiencia histórica} \rightarrow \text{pesos neuronales} \rightarrow \text{generalización futura}}.$$

15. Moderadores

Cada red tendrá un moderador independiente:

$$\mu_V, \qquad \mu_P, \qquad \mu_D.$$

La configuración inicial será

$$\mu_V = \mu_P = \mu_D = 0.$$

Ejemplo de interfaz:

```
--value-influence 0.25
--piece-influence 0.25
--depth-influence 0.20
```

Esto permite distinguir capacidad de aprendizaje de influencia efectiva sobre el juego.

16. Métricas obligatorias

La auditabilidad es un requisito funcional.

16.1. Métricas por red

Para cada red se registrará: ejemplos vistos, loss de entrenamiento, loss de validación, media y desviación de salida, magnitud media del gradiente, cambio medio de parámetros, influencia efectiva y composición por fuente.

16.2. ValueNet

Se comparará

$$MAE(V_S, Y^V)$$

con

$$MAE(V_{\text{eff}}, Y^V).$$

La mejora es

$$\Delta MAE = MAE(V_S, Y^V) - MAE(V_{\text{eff}}, Y^V).$$

Reducir sólo el loss interno no es evidencia suficiente.

16.3. PieceNet

Se medirán corrección media por pieza, distribución por fase, contribución al error total, estabilidad temporal y magnitud relativa respecto del valor base.

16.4. DepthNet

Se medirá

$$MAE(d_{\text{pred}}, d^*)$$

y además profundidad media solicitada y efectiva, porcentaje de posiciones extendidas, nodos adicionales, error de valor evitado y costo computacional por mejora.

16.5. Coincidencia con PGN

Sobre validación PGN no usada para entrenamiento:

$$A_{\text{top1}} = P(a_{\text{engine}} = a_{\text{expert}}),$$

$$A_{\text{topK}} = P(a_{\text{expert}} \in \text{TopK}).$$

16.6. Impacto sobre decisiones

Se registrarán la jugada base a_S y la jugada con redes a_L, midiendo

$$P(a_S \neq a_L).$$

16.7. Resultados por generación

Cada versión registrará cantidad de partidas, victorias, derrotas, tablas, fuente, modelo adversario cuando corresponda, duración media, nodos por jugada, profundidad media y tiempo por jugada.

17. Validación y conjuntos de prueba

Deben existir conjuntos congelados de validación: posiciones PGN no usadas en entrenamiento, posiciones tácticas y posicionales de referencia, partidas reproducibles con seeds fijas y enfrentamientos entre generaciones de modelos. No se permitirá evaluar progreso únicamente sobre datos de entrenamiento.

18. Criterios mínimos para afirmar que una red aprende

18.1. ValueNet

Debe disminuir el error de validación, mejorar $MAE(V_{\text{eff}}, Y^V)$ frente a SARGON, generalizar a datos no vistos y no degradar sistemáticamente el resultado del motor.

18.2. PieceNet

Sus correcciones deben ser estables, mejorar el target global, generalizar y no colapsar a correcciones extremas.

18.3. DepthNet

Debe predecir d^* mejor que una profundidad constante, reducir error de evaluación mediante extensiones, justificar el costo de nodos adicionales y permanecer dentro de límites de profundidad.

19. Auditoría obligatoria

Cada entrenamiento deberá permitir reconstruir: partida origen, versión del modelo, evaluación Sargon, correcciones neuronales, profundidad usada, target posterior al resultado, pérdidas producidas y modelo resultante. Cada modelo debe ser trazable a

$$\text{modelo anterior} + \text{partidas aprendidas} + \text{configuración} + \text{seed}$$

cuando sea posible.

20. Logging de consola

El sistema deberá reportar de forma compacta:

```
[game-start] source=worker-3 model=42
[game-end] source=worker-3 game=918 result=0-1 plies=83
[learn-start] game=918 model_in=42
[learn-value] samples=83 loss=...
[learn-piece] samples=83 loss=...
[learn-depth] samples=... loss=...
[learn-end] game=918 model_out=43 elapsed=...
[ack] source=worker-3 game=918
```

Cada productor reportará cuándo espera ACK y cuándo queda habilitado para una nueva partida.

21. Configuración mínima

La CLI deberá permitir, como mínimo:

```
--value-net <layers>
--piece-net <layers>
--depth-net <layers>
--value-influence <float>
--piece-influence <float>
--depth-influence <float>
--depth-base <int>
--depth-min <int>
--depth-max <int>
--depth-max-extra <int>
--gamma <float>
--lambda <float>
--pgn-weight <float>
--learning-rate-value <float>
--learning-rate-piece <float>
--learning-rate-depth <float>
```

La configuración exacta se almacenará en cada modelo.

22. Secuencia de activación

Aunque la infraestructura soporte las tres redes desde el comienzo, la activación funcional será escalonada.

22.1. Etapa 0

$$\mu_V = \mu_P = \mu_D = 0.$$

Debe coincidir con Sargon base.

22.2. Etapa 1

$$\mu_V > 0, \qquad \mu_P = 0, \qquad \mu_D = 0.$$

Se entrena y audita ValueNet.

22.3. Etapa 2

$$\mu_V > 0, \qquad \mu_P > 0, \qquad \mu_D = 0.$$

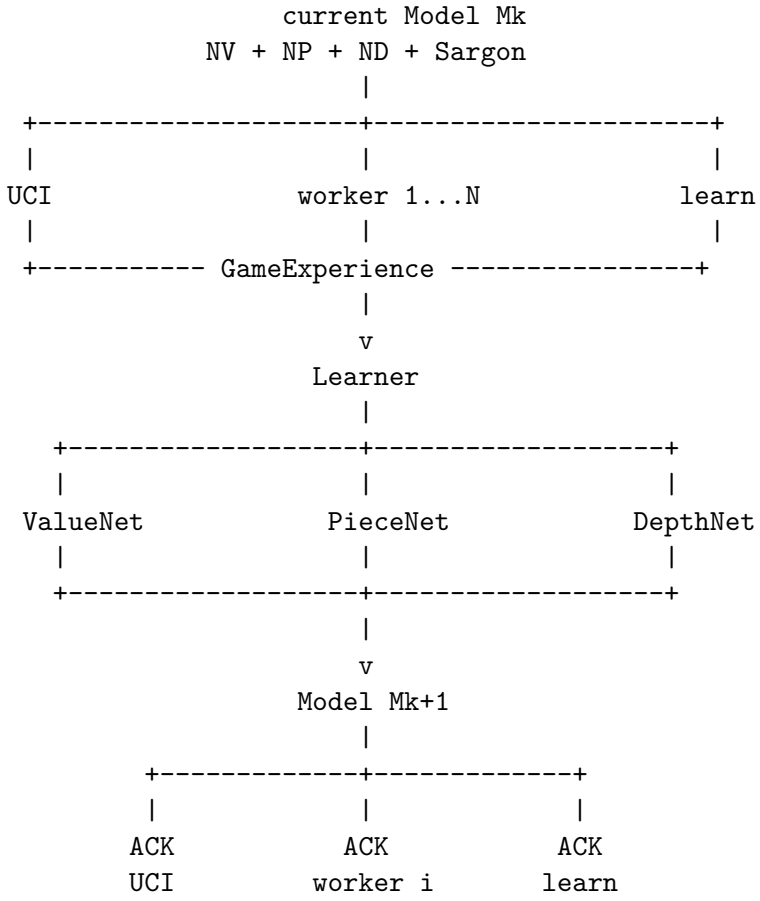
Se incorpora PieceNet.

22.4. Etapa 3

Se incorpora DepthNet con moderación y sólo extensiones:

$$\mu_D > 0, \qquad \Delta d \geq 0.$$

23. Arquitectura general



PGN entra como experiencia experta:

`PGN ---> parser ---> GameExperience(expert) ---> Learner`

24. Ciclo conceptual del conocimiento

Para valor:

posición → evaluación Sargon → análisis → target provisional → resultado → target temporal → redes → mod

Para PGN:

posición → jugada experta → comparación con alternativas → pérdida de preferencia → redes

Para profundidad:

$$V_1, \ldots, V_D \to d^* \to N_D \to \text{profundidad futura moderada}$$

25. Invariantes arquitectónicos

Se establecen como invariantes:

1. sólo existe un escritor lógico del modelo;
2. ningún productor modifica pesos;
3. una partida usa un único modelo;
4. una fuente no inicia una nueva partida antes del ACK;
5. cada worker posee su propia espera independiente;
6. todos los modelos son versionados;
7. todas las experiencias indican el modelo que las produjo;
8. todos los PGN son experiencia experta;
9. el conocimiento operativo reside en las redes, no en tablas por posición exacta;
10. la profundidad neuronal siempre está moderada y limitada;
11. ninguna mejora se considera demostrada sin validación fuera de muestra.

26. Objetivo experimental

La arquitectura debe permitir responder cuantitativamente:

1. £disminuye el error de valor?;
2. £cambia de manera estable la valoración contextual de piezas?;
3. £DepthNet identifica posiciones que realmente requieren más análisis?;
4. £las redes generalizan a posiciones no vistas?;
5. £los PGN aumentan la coincidencia con decisiones expertas?;
6. £los workers producen experiencia que mejora generaciones posteriores?;
7. £el modelo nuevo supera al anterior en métricas independientes?;
8. £el aumento de complejidad produce una mejora proporcional y verificable?

Si estas preguntas no pueden responderse con datos del propio sistema, la implementación no cumple esta especificación.

27. Conclusión

La arquitectura queda definida por cuatro ideas fundamentales. Primero, SARGON continúa siendo la línea base y proporciona estructura, evaluación y búsqueda. Segundo, el aprendizaje se concentra en tres redes neuronales especializadas:

$$\boxed{N_V, \quad N_P, \quad N_D}$$

para valor posicional, valor contextual de piezas y profundidad. Tercero, toda fuente de juego produce experiencia, pero sólo el learner actualiza el modelo:

$$\boxed{\text{todos producen; uno aprende}}.$$

Cuarto, el progreso debe ser demostrable mediante métricas reproducibles y validación independiente.

La especificación prioriza deliberadamente simplicidad, trazabilidad y capacidad de falsar la hipótesis de aprendizaje. El sistema debe poder demostrar no sólo que sus pesos cambian, sino que esos cambios reducen error, mejoran decisiones, utilizan mejor la profundidad y producen generaciones de juego progresivamente superiores.